Reviewer Pack — Minimal Geometric Entropy Bound for DPLL Proof Path Length v…

Entry a532431426bd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 18:25:45 UTC

Minimal Geometric Entropy Bound for DPLL Proof Path Length via Information Theoretic Invariants

Entry ID: `a532431426bd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 18:25:45 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

> For every instance φ of SAT, the geometric entropy H(φ) of its associated quantum state |ψ⟩, obtained by mapping φ to a binary Hermitian matrix A and then constructing the state |ψ⟩ = ∑|x⟩⟨x|/||A||_2, satisfies |H(φ)| ≤ 2 log(Δφ), where Δφ is the number of distinct satisfying assignments for φ.

Rationale (proposer's reasoning):

> Quantum information theory provides a framework to study the complexity of computation by associating quantum states with computational problems. Geometric entropy measures the uncertainty in the state and can be used to bound communication complexity, which is closely related to proof complexity. By linking geometric entropy to DPLL proof path length, this conjecture might reveal a new connection between quantum information theory and complexity theory.

Taxonomy category: `quantum_information_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `83479af3691bdcc5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given SAT instance φ, if the geometric entropy H(φ) satisfies |H(φ)| ≤ 2 log(Δφ), where Δφ is the number of distinct satisfying assignments for φ.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum information theory" AND "Boolean satisfiability" AND DPLL - "geometric entropy" AND "DPLL proof complexity" AND "satisfying assignments" - "Hermitian matrix" AND "binary Hermitian matrix" AND "proof path length"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.randint(-n, n) for _ in range(random.randint(1, n))]
            clauses.append(clause)
        return clauses

    def hermitian_matrix(clauses, n):
        A = [[0] * (2*n) for _ in range(2*n)]
        for clause in clauses:
            for lit in clause:
                row, col = abs(lit)-1, 2*(abs(lit)-1)
                if lit > 0:
                    A[row][col] += 1
                else:
                    A[row][col+1] += 1
        return A

    def geometric_entropy(A):
        eigenvalues = power_method(A, 100)  # Power method to find the largest eigenvalue
        H = -sum(eigenvalue * math.log2(eigenvalue) for eigenvalue in eigenvalues if eigenvalue >
        return H

    def power_method(matrix, max_iter=100):
        n = len(matrix)
        v = [random.random() for _ in range(n)]
```

```
    v /= sum(v)

    for _ in range(max_iter):
        Av = matrix @ v
        Av /= sum(Av)
        v = Av

    return Av

def delta_phi(clauses, n):
    assignments = list(itertools.product([-1, 1], repeat=n))
    valid_assignments = [assignment for assignment in assignments if all(lit * assignment[abs(
    return len(valid_assignments)

def is_square(matrix):
    n = len(matrix)
    return all(len(row) == n for row in matrix)

def normalize(matrix):
    n = len(matrix)
    norm = sum(sum(abs(x) for x in row) for row in matrix)
    return [[x / norm for x in row] for row in matrix]

def multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def transpose(matrix):
    n = len(matrix)
    return [[matrix[j][i] for j in range(n)] for i in range(n)]

def inverse(matrix):
    n = len(matrix)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    Aaug = [row + col for row, col in zip(matrix, I)]

    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(Aaug[r][i]))
        if Aaug[max_row][i] == 0:
            return None  # Singular matrix
        Aaug[i], Aaug[max_row] = Aaug[max_row], Aaug[i]

        for j in range(n):
            Aaug[i][j] /= Aaug[i][i]

        for k in range(n):
            if k != i:
                factor = Aaug[k][i]
                for j in range(2*n):
                    Aaug[k][j] -= factor * Aaug[i][j]
```

```
    return [row[n:] for row in Aaug]

def norm(matrix):
    n = len(matrix)
    return math.sqrt(sum(sum(x**2 for x in row) for row in matrix))

def normalize_rows(matrix):
    n = len(matrix)
    return [[x / sum(row) if sum(row) != 0 else 0 for x in row] for row in matrix]

def normalize_columns(matrix):
    n = len(matrix)
    return [[x / sum(col) if sum(col) != 0 else 0 for col in zip(*matrix)] for row in matrix]

def is_diagonalizable(matrix):
    n = len(matrix)
    eigenvalues, eigenvectors = diagonalize(matrix)
    return all(eigenvalue == eigenvector[0] for eigenvalue, eigenvector in zip(eigenvalues, ei

def diagonalize(matrix):
    n = len(matrix)
    A = normalize(matrix)
    Q = identity(n)
    T = copy(A)

    for i in range(n-1):
        if abs(T[i+1][i]) > 1e-10:
            c = (T[i][i] - T[i+1][i+1]) / (2 * T[i+1][i])
            s = math.sqrt(1 + c**2)
            c /= s
            s /= s

            Q_i = identity(n)
            Q_i[i][i] = c
            Q_i[i+1][i+1] = c
            Q_i[i][i+1] = -s
            Q_i[i+1][i] = s

            T_i = copy(T)
            T_i[i][i] = T_i[i][i] * c**2 + 2 * T_i[i][i+1] * c * s + T_i[i+1][i+1] * s**2
            T_i[i][i+1] = (T_i[i][i] - T_i[i+1][i+1]) * c * s + (T_i[i][i+1] + T_i[i+1][i+1])
            T_i[i+1][i] = T_i[i][i+1]
            T_i[i+1][i+1] = -T_i[i][i+1] * s**2 + 2 * T_i[i][i+1] * c * s + T_i[i+1][i+1] * c*

            Q = multiply(Q, Q_i)
            T = multiply(multiply(Q_i, T), transpose(Q_i))

    eigenvalues = [T[i][i] for i in range(n)]
    eigenvectors = [Q[:, i] for i in range(n)]

    return eigenvalues, eigenvectors

def identity(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def copy(matrix):
```

```
    n = len(matrix)
    return [[matrix[i][j] for j in range(n)] for i in range(n)]

def random_instance(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.randint(-n, n) for _ in range(random.randint(1, n))]
        clauses.append(clause)
    return clauses

def hermitian_matrix(clauses, n):
    A = [[0] * (2*n) for _ in range(2*n)]
    for clause in clauses:
        for lit in clause:
            row, col = abs(lit)-1, 2*(abs(lit)-1)
            if lit > 0:
                A[row][col] += 1
            else:
                A[row][col+1] += 1
    return A

def geometric_entropy(A):
    eigenvalues = power_method(A, 100)  # Power method to find the largest eigenvalue
    H = -sum(eigenvalue * math.log2(eigenvalue) for eigenvalue in eigenvalues if eigenvalue >
    return H

def power_method(matrix, max_iter=100):
    n = len(matrix)
    v = [random.random() for _ in range(n)]
    v /= sum(v)

    for _ in range(max_iter):
        Av = matrix @ v
        Av /= sum(Av)
        v = Av

    return Av

def delta_phi(clauses, n):
    assignments = list(itertools.product([-1, 1], repeat=n))
    valid_assignments = [assignment for assignment in assignments if all(lit * assignment[abs(
    return len(valid_assignments)

def is_square(matrix):
    n = len(matrix)
    return all(len(row) == n for row in matrix)

def normalize(matrix):
    n = len(matrix)
    norm = sum(sum(abs(x) for x in row) for row in matrix)
    return [[x / norm for x in row] for row in matrix]

def multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
```

```
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def transpose(matrix):
    n = len(matrix)
    return [[matrix[j][i] for j in range(n)] for i in range(n)]

def inverse(matrix):
    n = len(matrix)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    Aaug = [row + col for row, col in zip(matrix, I)]

    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(Aaug[r][i]))
        if Aaug[max_row][i] == 0:
            return None  # Singular matrix
        Aaug[i], Aaug[max_row] = Aaug[max_row], Aaug[i]

        for j in range(n):
            Aaug[i][j] /= Aaug[i][i]

        for k in range(n):
            if k != i:
                factor = Aaug[k][i]
                for j in range(2*n):
                    Aaug[k][j] -= factor * Aaug[i][j]

    return [row[n:] for row in Aaug]

def norm(matrix):
    n = len(matrix)
    return math.sqrt(sum(sum(x**2 for x in row) for row in matrix))

def normalize_rows(matrix):
    n = len(matrix)
    return [[x / sum(row) if sum(row) != 0 else 0 for x in row] for row in matrix]

def normalize_columns(matrix):
    n = len(matrix)
    return [[x / sum(col) if sum(col) != 0 else 0 for col in zip(*matrix)] for row in matrix]

def is_diagonalizable(matrix):
    n = len(matrix)
    eigenvalues, eigenvectors = diagonalize(matrix)
    return all(eigenvalue == eigenvector[0] for eigenvalue, eigenvector in zip(eigenvalues, ei

def diagonalize(matrix):
    n = len(matrix)
    A = normalize(matrix)
    Q = identity(n)
    T = copy(A)

    for i in range(n-1):
        if abs(T[i+1][i]) > 1e-10:
```

```
            c = (T[i][i] - T[i+1][i+1]) / (2 * T[i+1][i])
            s = math.sqrt(1 + c**2)
            c /= s
            s /= s

            Q_i = identity(n)
            Q_i[i][i] = c
            Q_i[i+1][i+1] = c
            Q_i[i][i+1] = -s
            Q_i[i+1][i] = s

            T_i = copy(T)
            T_i[i][i] = T_i[i][i] * c**2 + 2 * T_i[i][i+1] * c * s + T_i[i+1][i+1] * s**2
            T_i[i][i+1] = (T_i[i][i] - T_i[i+1][i+1]) * c * s + (T_i[i][i+1] + T_i[i+1][i+1])
            T_i[i+1][i] = T_i[i][i+1]
            T_i[i+1][i+1] = -T_i[i][i+1] * s**2 + 2 * T_i[i][i+1] * c * s + T_i[i+1][i+1] * c*

            Q = multiply(Q, Q_i)
            T = multiply(multiply(Q_i, T), transpose(Q_i))

    eigenvalues = [T[i][i] for i in range(n)]
    eigenvectors = [Q[:, i] for i in range(n)]

    return eigenvalues, eigenvectors

def identity(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def copy(matrix):
    n = len(matrix)
    return [[matrix[i][j] for j in range(n)] for i in range(n)]

def random_instance(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.randint(-n, n) for _ in range(random.randint(1, n))]
        clauses.append(clause)
    return clauses

def hermitian_matrix(clauses, n):
    A = [[0] * (2*n) for _ in range(2*n)]
    for clause in clauses:
        for lit in clause:
            row, col = abs(lit)-1, 2*(abs(lit)-1)
            if lit > 0:
                A[row][col] += 1
            else:
                A[row][col+1] += 1
    return A

def geometric_entropy(A):
    eigenvalues = power_method(A, 100)  # Power method to find the largest eigenvalue
    H = -sum(eigenvalue * math.log2(eigenvalue) for eigenvalue in eigenvalues if eigenvalue >
    return H

def power_method(matrix, max_iter=100):
```

```
        n = len(matrix)
        v = [random.random() for _ in range(n)]
        v /= sum(v)

        for _ in range(max_iter):
            Av = matrix @ v
            Av /= sum(Av)
            v = Av

        return Av

    def delta_phi(clauses, n):
        assignments = list(itertools.product([-1, 1], repeat=n))
        valid_assignments = [assignment for assignment in assignments if all(lit * assignment[abs(
        return len(valid_assignments)

    def is_square(matrix):
        n = len(matrix)
        return all(len(row) == n for row in matrix)

    def normalize(matrix):
        n = len(matrix)
        norm = sum(sum(abs(x) for x in row) for row in matrix)
        return [[x / norm for x in row] for row in matrix]

    def multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def transpose(matrix):
        n = len(matrix)
        return [[matrix[j][i] for j in range(n)] for i in range(n)]
# ... [truncated, full source in replay tarball]
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fb49b38.py", line 125, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fb49b38.py", line 97, in run_trial
    delta_phi = len([assignment for assignment in itertools.product([-1, 1], repeat=n) if all(lit * ass
1-1] >= 0 for lit in clauses)])
                                                                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fb49b38.py", line 97, in <genexpr>
```

```
   delta_phi = len([assignment for assignment in itertools.product([-1, 1], repeat=n) if all(lit * ass
1-1] >= 0 for lit in clauses)])
                                                                                    ^^^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution due to a TypeError, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15190
2	preregistration	ollama_remote	glm4:latest	0	0	9793
3	novelty	ollama_remote	glm4:latest	0	0	9411
4	novelty	ollama_remote	glm4:latest	0	0	9152
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	62934
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15500
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16173
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	87531
9	judge	ollama_remote	glm4:latest	0	0	31665

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 257349 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a532431426bd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a532431426bd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a532431426bd.tar.gz` (if generated)